Mini Boolean Retrieval Information System

Create a small-scale search engine for a specific collection of documents (e.g., a set of Wikipedia articles, a university's course catalog). This project would involve at least building a crawler to gather data, one or more persistent inverted indexes (Standard Inverted Index, BiWord Index, Phrase Index, Positional Index, Permuterm Index, K-Gram Index, Parametric Index, Zone Index, Tiered Index) for efficient searching, and a ranking algorithm like TF-IDF or BM25. The project must make use of the data structures described during lectures (Linked List, B-Tree, Binary Heap, Trie, Binary Tree) and should implement one or more further functionalities:

- Conjunctive query, Optimized Conjunctive Query, Disjunctive query, Phrase query
- WildCard Query
- Proximity Operators
- Linguistic Preprocessing: Normalization, Stop List, Stop Word, Stemming
- Augmentation
- Skip List
- Spelling Correction (Edit Distance, K-gram overlap, Soundex Algorithm)
- Distributed Indexing
- Posting List Compression and Dictionary Compression
- Page Similarity (Fingerprint, Signature)

The paper must describe at least:

- the document collection
- the data structures used in the project to implement Dictionary and Posting Lists
- all the implemented functionality used during indexing and searching
- strengths and weakness of the implementation

Mini Semantic Retrieval Information System

Build a search engine that retrieves documents based on their semantic relevance to a query, not just on exact keyword matches. You must represent both the query and documents as vectors and then rank documents by their cosine similarity to the query vector. Vector components can be computed using the Term Count (tc), the Term Frequency (tf), the TF_IDF model or by averaging the Word2Vec or FastText vectors of the words they contain, or Doc2Vec. The project can also implement one or more functionalities listed in the previous section.

The paper must describe at least:

- the document collection
- how vectors have been computed
- all the implemented functionality used during indexing and searching

- strengths and weakness of the implementation

Experimental Projects

- **Image Search Engine**: Develop a search engine that retrieves images based on text queries. This would require using a content-based image retrieval (CBIR), which involves extracting features like color, texture, and shape.
- **Sentiment Analysis**: Use a pre-trained Word2Vec model to get word embeddings for movie reviews or customer feedback. You can then average the word vectors in a sentence to get a document vector, which can be used as input for a machine learning model (like a support vector machine or a simple neural network) to classify the text as positive, negative, or neutral.
- **Document Clustering**: Group a collection of documents (e.g., news articles, scientific papers) into clusters based on their content. You would represent each document as a vector using Word2Vec and then apply a clustering algorithm like K-Means to group the documents with similar semantic meanings
- **Topic Classification:** Train a FastText or a Word2Vec model on a specialized corpus, like medical journals, legal documents, or financial reports. These domain-specific embeddings will be more effective for tasks within that field than generic, pre-trained models.
- **Spam Detection:** Create a classifier to distinguish between spam and non-spam emails or messages. FastText's speed and efficiency make it suitable for real-time applications like this.
- **Next Word Prediction**: By using Word Embedding, build a Next Word Predictor which is able to predict the next word of a sentence given the context of the sentence